PriceIQ

AI-Powered Electronics Price Comparison for Indian Consumers

Software Requirements Specification (SRS)

Document Version 1.0

Project Type:	BTech Final Year Project
Department:	Computer Science and Engineering
Academic Year:	2025–2026
Prepared By:	Adrij Bhadra
Date:	April 2026
Document Status:	Final

Contents

1 Introduction **3**

1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Definitions, Acronyms, and Abbreviations . . . 4
1.4 Overview of Document . . . 5

2 Overall Description **6**

2.1 Product Perspective . . . 6
2.2 Product Functions . . . 6
2.3 User Classes and Characteristics . . . 7
2.4 Operating Environment . . . 7
2.5 Design and Implementation Constraints . . . 8

3 Functional Requirements **9**

3.1 FR-001: Live Search . . . 9
3.2 FR-002: Product Identity Matching . . . 10
3.3 FR-003: Price Comparison Display . . . 11
3.4 FR-004: AI Purchase Recommendation . . . 12
3.5 FR-005: Price History Tracking . . . 13
3.6 FR-006: Batch Scraping . . . 13
3.7 FR-007: Database Text Search and Filtering . . . 14
3.8 FR-008: On-Demand Price Refresh . . . 15
3.9 FR-009: Administrative Dashboard . . . 16
3.10 FR-010: Dark Mode UI Theme . . . 16

4 Non-Functional Requirements **18**

4.1 Performance Requirements . . . 18
4.2 Reliability Requirements . . . 18
4.3 Security Requirements . . . 19
4.4 Usability Requirements . . . 20

4.5 Maintainability Requirements 20

5 External Interface Requirements 22

5.1 User Interface Requirements 22

5.2 Hardware Interfaces 22

5.3 Software Interfaces 22

5.4 Communication Interfaces 23

6 Data Requirements 24

6.1 Logical Data Model 24

6.2 Data Retention and Integrity 25

7 Constraints, Assumptions, and Dependencies 26

7.1 Constraints 26

7.2 Assumptions 26

7.3 Dependencies 27

Chapter 1

Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document formally describes the functional and non-functional requirements for **PriceIQ**, an AI-powered electronics price comparison platform designed for Indian consumers. The document is intended for the following audiences:

- Academic evaluators and examiners assessing the BTech final-year project.
- The developer(s) implementing and maintaining the system.
- Any future contributors who extend the platform.

This SRS is the authoritative reference for what the system *must* do, how it must behave, and the constraints under which it operates. It does not describe implementation details, which are covered in the System Design Document.

1.2 Scope

PriceIQ is a web-based application that allows users to search for consumer electronics and compare real-time prices from three major Indian online retailers: Amazon India (`amazon.in`), Flipkart (`flipkart.com`), and Vijay Sales (`vijaysales.com`). The system additionally generates AI-driven purchase recommendations using the Groq LLaMA-3 large language model.

The application is scoped to the Indian market (prices in Indian Rupees, INR), and covers the following product categories: Smartphones, Laptops, Headphones, Televisions, Tablets, Cameras, and Wearables.

In scope:

- User search and live scraping across three platforms.

- Price comparison display with source-level detail.
- Price history tracking across scrape cycles.
- AI-generated product summary and purchase recommendation.
- Batch scraping of a curated product catalogue.
- Administrative dashboard with KPI metrics.

Out of scope:

- E-commerce transaction or cart functionality.
- User account management or personalisation.
- Price alert notifications or email/SMS delivery.
- Support for non-Indian retailers or non-INR currencies.
- Mobile (iOS/Android) native applications.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
PriceIQ	The name of the system being specified.
SRS	Software Requirements Specification.
API	Application Programming Interface.
LLM	Large Language Model.
Groq	A hardware-accelerated AI inference provider; used to run LLaMA-3.
LLaMA-3	Open-weight large language model developed by Meta.
ORM	Object-Relational Mapper; specifically Prisma 7 in this project.
SSR	Server-Side Rendering (Next.js feature).
ASIN	Amazon Standard Identification Number, a unique product identifier on Amazon.
Playwright	A browser automation library used to scrape Flipkart.
Slug	A URL-safe lowercase string uniquely identifying a product, e.g., `iphone-15-128gb`.
INR	Indian Rupee, the currency used throughout the system.
FR	Functional Requirement.
NFR	Non-Functional Requirement.
MRP	Maximum Retail Price, the government-mandated ceiling price on Indian goods.

1.4 Overview of Document

The remainder of this document is organised as follows:

- **Chapter 2** provides an overall description of the product, including product perspective, key functions, user classes, and operating environment.
- **Chapter 3** specifies all functional requirements, grouped into ten major feature areas with detailed use-case descriptions.
- **Chapter 4** defines non-functional requirements covering performance, reliability, security, usability, and maintainability.
- **Chapter 5** describes external interface requirements including the user interface, third-party API interfaces, and communication protocols.
- **Chapter 6** presents the system's data model and persistence requirements.
- **Chapter 7** lists assumptions, dependencies, and known constraints.

Chapter 2

Overall Description

2.1 Product Perspective

PriceIQ is a standalone web application operating in a client-server model. It is not a sub-system of any larger platform; however, it depends on three external data sources (Amazon India, Flipkart, Vijay Sales) and one external AI inference service (Groq API) to deliver its core value.

The system fills a gap in the Indian consumer market: existing price-comparison services (e.g., BuyHatke, PriceDekho) do not provide AI-generated analysis tailored to Indian buying priorities such as warranty support, seller trust, and delivery speed. PriceIQ combines real-time scraping with LLM inference to address this gap.

(Refer to the System Design Document for the full architecture diagram.)

2.2 Product Functions

The major functions provided by PriceIQ are summarised below:

1. **Live Search:** On receiving a user query, the system concurrently scrapes Amazon India, Flipkart, and Vijay Sales and returns normalised price data.
2. **Product Identity Matching:** A scoring engine parses the query into brand, model, storage, and variant tokens and scores each retailer candidate to prevent wrong-variant or accessory listings.
3. **Price Comparison:** Aggregated listings are displayed in a structured comparison table showing price, rating, stock status, and a "best deal" highlight.
4. **AI Recommendation:** The Groq LLaMA-3 model analyses the available listings and generates a concise purchase recommendation tailored to Indian consumers.

5. **Price History:** Every scrape cycle appends a timestamped price record, enabling users to observe trends via a line chart.
6. **Batch Scraping:** An administrative endpoint runs a full scrape of a curated product catalogue and populates the database.
7. **Search:** Users can query the product database with text search and apply category, source, and sort filters.
8. **Dashboard:** Administrators can view system KPIs including total products, listing counts by source, recent searches, and category distribution.

2.3 User Classes and Characteristics

User Class	Frequency	Characteristics
General Consumer	High	Indian consumer looking to compare prices before purchasing electronics. Non-technical. Interacts via search bar and product page.
Power User	Medium	Tech-savvy user who reads AI analysis, checks price history charts, and uses category/sort filters.
Administrator	Low	Developer or project maintainer who triggers batch scraping, monitors the dashboard, and manages the product catalogue.

2.4 Operating Environment

The system operates in the following environment:

- **Server OS:** Linux (Docker container) or Windows 11 (development).
- **Runtime:** Node.js 20+, Next.js 16 App Router.
- **Database:** PostgreSQL 16 via Docker Compose.
- **Browser:** Any modern Chromium, Firefox, or Safari-based browser (desktop and mobile).
- **Network:** Outbound HTTPS access to `amazon.in`, `flipkart.com`, `vsprod.vijaysales.com`, and `api.groq.com`.

2.5 Design and Implementation Constraints

- The system must use Next.js 16 App Router architecture to comply with the project's approved technology stack.
- All monetary values are stored and displayed in INR only.
- Flipkart scraping requires a headless Chromium browser (Playwright) and cannot be replaced by a plain HTTP client due to the site's bot-detection measures.
- The Groq API key must be stored as an environment variable and never committed to version control.
- The database schema is managed by Prisma 7; the `DATABASE_URL` must be declared in `prisma.config.ts` (not `schema.prisma`).
- The batch scrape endpoint must be protected by a shared secret (`SCRAPE_SECRET`) transmitted via the `x-scrape-secret` HTTP header.

Chapter 3

Functional Requirements

Each requirement is given a unique identifier in the format **FR-NNN**. Priority levels: **M** = Must Have, **S** = Should Have, **C** = Could Have.

3.1 FR-001: Live Search

ID: FR-001 **Priority:** M **Actor:** General Consumer

Description: When a user submits a search query that returns no results from the local database, the system must automatically initiate a live search by concurrently scraping Amazon India, Flipkart, and Vijay Sales.

Field	Detail
Pre-conditions	Application is running; user has entered a non-empty search query; no matching product exists in the database.

Field	Detail
Main Flow	1. User submits query via the search bar. 2. System queries the database; result count is zero. 3. `LiveSearchTrigger` component fires a POST to `/api/search/live`. 4. Server calls `liveSearch(query)`: Vijay Sales (GraphQL) and Amazon (HTTP) are scraped in parallel; Flipkart (Playwright) runs sequentially after. 5. Each platform returns up to one `NormalizedProduct`. 6. System upserts a canonical `Product` record and one `ProductListing` per platform. 7. A `PriceHistory` entry is recorded for each listing. 8. System redirects the user to `/product/{id}`.
Alternative Flow A	Platform scrape fails (timeout, bot block): that platform is skipped; remaining platforms' results are still returned.
Alternative Flow B	No platform returns an accepted candidate: system returns an error message to the user suggesting they refine the query.
Post-conditions	At least one `ProductListing` record is stored; user is shown the product comparison page.

3.2 FR-002: Product Identity Matching

ID: FR-002 **Priority:** M **Actor:** System (internal)

Description: The system must use a structured scoring engine to verify that a retailer-returned product title matches the user's search intent before storing or displaying the result.

Field	Detail
Pre-conditions	A raw candidate product title and a parsed query object are available.

Field	Detail
Main Flow	1. `parseQuery()` extracts brand, model tokens, storage token (e.g., 128GB), and variant token (e.g., Ultra) from the query. 2. `scoreCandidate()` evaluates the candidate title: a) Hard-reject accessory keywords (case, cover, charger, etc.). b) Hard-reject renewed/refurbished listings unless query requests them. c) Hard-reject if no model tokens match. d) Hard-reject storage or variant conflicts. e) Apply soft score bonuses for brand match (+30), model token coverage (+40 max), storage match (+20), variant match (+20). f) Apply soft penalties for combos, unexpected variants, missing storage, missing brand. 3. A candidate is accepted if score ≥ 40 (when model tokens exist) or score ≥ 20 (vague query). 4. The highest-scoring accepted candidate is selected; ties are broken by review count.
Post-conditions	Only products that semantically match the query are stored in the database.

3.3 FR-003: Price Comparison Display

ID: FR-003 **Priority:** M **Actor:** General Consumer

Description: The product detail page must display a structured comparison of all available platform listings for a product.

Field	Detail
Pre-conditions	A product with at least one listing exists in the database.

Field	Detail
Main Flow	1. User navigates to `/product/{id}`. 2. System loads `Product` with all `ProductListing` records. 3. Listings are sorted: in-stock first, then by ascending price. 4. The cheapest in-stock listing is highlighted as "Best Deal". 5. Price differences from the best deal are shown as percentage. 6. Each listing shows: platform name, price (INR), star rating, review count, MRP (Vijay Sales only), stock status, and an outbound link.
Alternative Flow	A listing is out of stock: it is shown last with a grey "Out of Stock" badge.
Post-conditions	User can see prices from all scraped platforms on one page.

3.4 FR-004: AI Purchase Recommendation

ID: FR-004 **Priority:** M **Actor:** General Consumer, System

Description: The system must generate an AI-driven purchase recommendation for each product using the Groq LLaMA-3 inference API.

Field	Detail
Pre-conditions	Product has at least one listing; Groq API key is configured.

Field	Detail
Main Flow	1. After live search completes, server calls `analyzeProduct()` with product name, category, domain, and all listings. 2. Groq LLaMA-3.3-70b model receives a structured prompt requesting JSON output with: `summary`, `recommendation`, `bestSource`, `confidenceNote`. 3. Response is parsed; if JSON is malformed, a regex-based text fallback extracts the fields. 4. Result is stored in `AIAnalysis`; cached for 24 hours. 5. User sees the analysis on the "AI Analysis" tab of the product page.
Alternative Flow	Groq API call fails: product is still saved; AI tab shows a "Refresh" button so the user can retry.
Post-conditions	An `AIAnalysis` record is stored with a 24-hour TTL.

3.5 FR-005: Price History Tracking

ID: FR-005 **Priority:** S **Actor:** System, Power User

Description: Every scrape event must append a timestamped price record to the `PriceHistory` table, enabling trend visualisation.

Field	Detail
Pre-conditions	A scrape event (live or batch) has completed with at least one accepted listing.
Main Flow	1. For each accepted `NormalizedProduct`, system creates a `PriceHistory` row with: `productId`, `source`, `price`, `recordedAt` (UTC timestamp). 2. Rows are never updated or deleted (append-only). 3. Product page renders a line chart via Recharts querying `/api/price-history/{id}`.
Post-conditions	Chart on product page reflects all historical price points.

3.6 FR-006: Batch Scraping

ID: FR-006 **Priority:** M **Actor:** Administrator

Description: An authenticated HTTP endpoint must trigger a full scrape of the curated product catalogue (`products.ts`) across all three platforms.

Field	Detail
Pre-conditions	Request includes a valid `x-scrape-secret` header.
Main Flow	1. POST to `/api/scrape` with header `x-scrape-secret: <SCRAPE_SECRET>`. 2. Server validates the secret against `process.env.SCRAPE_SECRET`. 3. `runAllScrapers()` is called: iterates over 4 products × 3 sources. 4. Results are grouped by canonical slug, upserted into `Product` and `ProductListing`, and price history is recorded. 5. JSON response returns the count of upserted records.
Alternative Flow	Missing or wrong secret: server responds 403 Forbidden immediately.
Post-conditions	Database reflects latest prices for all catalogue products.

3.7 FR-007: Database Text Search and Filtering

ID: FR-007 **Priority:** M **Actor:** General Consumer

Description: The search results page must support full-text search of the product database with category filtering, source filtering, and multiple sort options.

Field	Detail
Pre-conditions	Database contains at least one product.

Field	Detail
Main Flow	1. User enters query in the search bar and optionally selects filters. 2. GET `/api/search?q={query}&category={cat}&sort={sort}` is issued. 3. Database performs case-insensitive match on `name`, `brand`, `category`, and `description` fields. 4. Optional filters: category (exact match), source (listing source). 5. Sort options: price ascending/descending, rating, source count (default). 6. Search query is logged to `SearchLog`. 7. Results rendered as a grid of `ProductCard` components.
Alternative Flow	Zero results found: `LiveSearchTrigger` is shown (see FR-001).
Post-conditions	Matching products displayed; query logged.

3.8 FR-008: On-Demand Price Refresh

ID: FR-008 **Priority:** S **Actor:** Power User

Description: From the product detail page, a user must be able to trigger a re-scrape of all three platforms for the displayed product.

Field	Detail
Pre-conditions	Product page is open with an existing product record.
Main Flow	1. User clicks “Refresh Prices” button. 2. Frontend calls `/api/product/{id}/refresh`. 3. Server re-runs `liveSearch()` with the product’s canonical name. 4. Updated listings are upserted; new price history rows are created. 5. Page re-renders with updated data; toast notification confirms success.

Field	Detail
Post-conditions	Listings and price history updated; AI analysis cache invalidated.

3.9 FR-009: Administrative Dashboard

ID: FR-009 **Priority:** S **Actor:** Administrator

Description: A dedicated dashboard page must display system-wide KPI metrics including product count, listing distribution by source, recent search activity, and category breakdown.

Field	Detail
Pre-conditions	At least one product and one search log entry exist.
Main Flow	1. User navigates to `/dashboard`. 2. Page calls `/api/dashboard` which aggregates: total products, total listings per source, recent search queries, and category distribution. 3. KPI cards, a source distribution pie chart (Recharts), a category bar chart, and a recent-searches table are rendered.
Post-conditions	Administrator can assess system coverage at a glance.

3.10 FR-010: Dark Mode UI Theme

ID: FR-010 **Priority:** C **Actor:** General Consumer

Description: The application UI must support a dark colour theme that is applied by default and persists across sessions.

Field	Detail
Pre-conditions	User accesses any page.

Field	**Detail**
Main Flow	1. `ThemeProvider` (next-themes) initialises with theme = "dark". 2. Theme is stored in `localStorage` and restored on subsequent visits. 3. All Tailwind CSS and Radix UI components respond to the `dark:` variant class.
Post-conditions	UI renders in dark mode consistently across all pages.

Chapter 4

Non-Functional Requirements

4.1 Performance Requirements

ID	Requirement	Metric / Acceptance Criterion
NFR-P01	Live search latency	Total live search (all 3 platforms) must complete within **60 seconds** under normal network conditions. Flipkart (Playwright) dominates latency.
NFR-P02	Cached search latency	Database text search must return results within **500 ms** for a product catalogue of up to 1 000 products.
NFR-P03	Product page load	SSR product page must reach Time-to-First-Byte within **2 seconds** on a standard broadband connection.
NFR-P04	AI analysis latency	Groq API call must complete within **8 seconds** at the model `llama-3.3-70b-versatile` with a 450-token output limit.
NFR-P05	Concurrent users	System must handle at least **5 concurrent live search requests** without returning server errors, given the single-instance deployment model.

4.2 Reliability Requirements

ID	Requirement	Metric / Acceptance Criterion
NFR-R01	Partial scrape failure	If one or two platform scrapers fail, the remaining results must still be saved and shown to the user (graceful degradation).
NFR-R02	AI failure isolation	If the Groq API is unreachable, the product page must still render price data; the AI tab shows a retry button.
NFR-R03	Database uptime	The PostgreSQL instance (Docker) must recover automatically on container restart. Data must survive container restarts via Docker volume.
NFR-R04	Bot detection handling	If Amazon.in returns a 503 or CAPTCHA, the Amazon scraper must log a warning and return null rather than throwing an unhandled exception.

4.3 Security Requirements

ID	Requirement	Metric / Acceptance Criterion
NFR-S01	Scrape endpoint protection	POST `/api/scrape` must reject any request whose `x-scrape-secret` header does not match `process.env.SCRAPE_SECRET`. Response: 403 Forbidden.
NFR-S02	API key security	`GROQ_API_KEY`, `DATABASE_URL`, and `SCRAPE_SECRET` must be stored only in `.env.local` (or Docker environment variables) and must never be embedded in client-side code.
NFR-S03	Input sanitisation	Search query parameters must be passed to the Prisma ORM as parameterised values, preventing SQL injection.

ID	Requirement	Metric / Acceptance Criterion
NFR-S04	No sensitive data in responses	API responses must never expose environment variable values, database connection strings, or internal stack traces to the client.

4.4 Usability Requirements

ID	Requirement	Metric / Acceptance Criterion
NFR-U01	Mobile responsiveness	All pages must be fully usable on a 375 px wide viewport (iPhone SE) using Tailwind CSS responsive utilities.
NFR-U02	Loading feedback	The `LiveSearchTrigger` component must display an animated per-platform progress indicator while scraping is in progress.
NFR-U03	Error messaging	All user-visible errors must be described in plain English with actionable guidance (e.g., "No results found. Try a more specific query.").
NFR-U04	Accessibility	All interactive controls must have visible focus states and ARIA labels sufficient to pass basic keyboard navigation.

4.5 Maintainability Requirements

ID	Requirement	Metric / Acceptance Criterion
NFR-M01	TypeScript strict mode	All source files must compile with `strict: true` in `tsconfig.json` without type errors.

ID	Requirement	Metric / Acceptance Criterion
NFR-M02	Scraper extensibility	Adding a new retailer must require changes to at most three files: a new scraper module, `index.ts`, and `live.ts`. The `matcher.ts` scoring engine and `types.ts` interface must not require modification.
NFR-M03	Product catalogue extension	Adding a new product to the curated catalogue must require only adding one entry to `products.ts`; no other code changes.

Chapter 5

External Interface Requirements

5.1 User Interface Requirements

- The application must provide a single-page hero section on the home route (`/`) with a prominently placed search bar.
- The search bar must support auto-submission on Enter key press and display search suggestions fetched from `/api/search/suggestions`.
- The product detail page (`/product/[id]`) must organise content into two tabs: *Price Comparison* and *AI Analysis*.
- All pages must include a persistent navigation bar with the PriceIQ brand logo, a search field, and a link to the dashboard.
- Toast notifications (Sonner library) must be used for scrape-trigger and refresh-success feedback; no modal dialogs.

5.2 Hardware Interfaces

The system has no direct hardware interface requirements beyond standard web server provisioning. The Playwright headless Chromium browser requires a minimum of 512 MB RAM per browser instance.

5.3 Software Interfaces

Interface	Version	Description
Groq API	`groq-sdk` 1.1.2	REST-over-HTTPS chat completions API. Model: `llama-3.3-70b-versatile`. Requires `GROQ_API_KEY` in environment.
Vijay Sales GraphQL	Internal	GraphQL endpoint at `vsprod.vijaysales.com/graphql`. Publicly accessible; no auth required. Requires a cache-bypass timestamp parameter (`tp`).
Amazon India	Public HTML	Standard HTTPS fetch of `amazon.in/s` (search) and `amazon.in/dp/{ASIN}` (product page). Subject to bot-detection blocking (503 responses).
Flipkart	Playwright DOM	Headless Chromium navigates `flipkart.com/search`. Anti-detection headers and navigator spoof required.
Prisma ORM	7.7.0	Prisma client with `@prisma/adapter-pg` (PostgreSQL adapter).
PostgreSQL	16	Relational database running in Docker. Accessed via connection string.
Next.js	16.2.4	Full-stack React framework. Provides API routes, SSR, and static assets.

5.4 Communication Interfaces

- All client-server communication uses **HTTPS** (TLS 1.2+) in production; HTTP is acceptable in local development only.
- API routes follow **REST** conventions: POST for mutations, GET for queries, standard HTTP status codes.
- Scraper communication with external sites uses **HTTPS** only.
- Groq API requests are authenticated via **Bearer token** in the `Authorization` header.

Chapter 6

Data Requirements

6.1 Logical Data Model

The system persists data in five relational tables. Their logical relationships are described here; the physical schema diagram appears in the System Design Document.

Entity	Description
Product	Represents one canonical electronics product (e.g., "Apple iPhone 15 128GB"). Identified by a unique slug. One product can have multiple listings and price history entries.
ProductListing	Represents the presence of a product on a specific retail platform (Amazon, Flipkart, or Vijay Sales) with current price, rating, stock status, and source URL. Unique per (productId, source) pair.
PriceHistory	An append-only record of price observations over time. One row per (product, source, scrape event). Used to generate price-trend charts.
AIAnalysis	Stores the Groq LLM output for a product: summary, recommendation, best-source name, and a confidence note. Refreshed every 24 hours.
SearchLog	Logs each user search query with a result count and timestamp. Used to populate the dashboard "Recent Searches" table.

6.2 Data Retention and Integrity

- `Product.slug` carries a `UNIQUE` constraint enforced by the database; Prisma upsert operations use it as the conflict key.
- `ProductListing` has a composite unique constraint on `(productId, source)` ensuring at most one listing per source per product.
- `PriceHistory` rows are never updated or deleted; they form an immutable audit trail.
- `AIAnalysis` has no unique constraint; the application uses `findFirst` followed by `update` or `create` to manage at most one active analysis per product.
- `SearchLog` is retained indefinitely for analytics; no PII is stored (no user IDs or IP addresses).
- All timestamps are stored in UTC and displayed in the user's local time zone via JavaScript's `Intl` API.

Chapter 7

Constraints, Assumptions, and Dependencies

7.1 Constraints

1. **Amazon Bot Detection:** Amazon India returns HTTP 503 or 500 for programmatic fetch requests, regardless of User-Agent spoofing. The Amazon scraper will silently return null in production and this is a known limitation. Fixing this would require using the Amazon Product Advertising API (requires seller account) or a rotating proxy service (out of scope for this project).
2. **Flipkart Rate Limiting:** The Playwright scraper introduces a significant latency overhead (25–45 seconds per product) due to full browser initialisation and JavaScript rendering.
3. **Vijay Sales Catalogue:** Vijay Sales does not stock all electronics models. Products unavailable in their catalogue return null and are excluded from comparisons.
4. **LLM Token Limit:** The Groq API call is capped at 450 output tokens; very long product descriptions may be truncated.
5. **Single-Region Scope:** All prices are in INR and reflect the Indian market only. Currency conversion is outside scope.

7.2 Assumptions

1. The developer has a valid Groq API key with sufficient quota for the expected usage (50 analysis requests per day).
2. Docker and Docker Compose are installed on the deployment host.
3. The host machine has at least 4 GB RAM to run PostgreSQL, Next.js, and a

Playwright Chromium instance concurrently.

4. The `.env.local` file is populated with `DATABASE_URL`, `GROQ_API_KEY`, and `SCRAPE_SECRET` before running the application.
5. Vijay Sales and Flipkart do not significantly change their DOM or API structure during the project evaluation period.

7.3 Dependencies

Dependency	**Version**	**Purpose**
Next.js	16.2.4	Full-stack React framework
React	19.2.4	UI rendering library
Prisma	7.7.0	Database ORM
`@prisma/adapter-pg`	7.7.0	PostgreSQL driver adapter for Prisma 7
Playwright	1.59.1	Headless browser for Flipkart scraping
`groq-sdk`	1.1.2	Groq LLM API client
TailwindCSS	4	Utility-first CSS framework
Recharts	3.8.1	Chart components for price history visualisation
Radix UI	various	Accessible headless UI primitives
Sonner	latest	Toast notification library
next-themes	latest	Dark mode theme management
Docker / Compose	24+ / 2.x	Container runtime for PostgreSQL

Revision History

Version	Date	Author	Changes
1.0	April 2026	Adrij Bhadra	Initial release. Covers all 10 functional requirements, NFRs, external interfaces, data model, and constraints.